



CS214: Data Structures

Assignment 1

<i>Name</i>	<i>ID</i>
<i>Aliaa Fares Mostafa</i>	<i>20242449</i>
<i>Hagar Ahmed Said Hassan Elsayed</i>	<i>20242446</i>
<i>Maryam Amr Abbas Metwaly</i>	<i>20242444</i>
<i>Sara Ali Refaey Abdelhamed</i>	<i>20242148</i>

Part A: Complexity Analysis

```
1) int hello_recursive(int n) {  
    if (n <= 1) return 1;  
    return hello_recursive(n-  
1) + hello_recursive(n-2);  
}
```

- For “if (n <=1) return 1: O(1) for if condition
- Grows exponentially each recursive call forming a Tree-like structure For example trace for *hello_recursive(3)* :

Call(1): n = 3, so returns hello_recursive(2)+ hello_recursive(1)
[2Calls]

Call(2): n =2,
hello_recursive(2)-> hello_recursive(1)+
hello_recursive(0)
(reached base case)
hello_recursive(1)-> reached base case
[3 Calls]

Conclusion: As hello_recursive(3) resulted in a total of 5 recursive calls + 1 original =6 ,we can shows that it has a binary tree like structure where each node makes up to max two recursive calls (n-1 and n-2)[2 children]. The tree's maximum depth is n so the maximum nodes at the bottom level is 2^n . Time-complexity(worst-case): $O(2^n)$

```

2) void test(int n) {
    • For int i = n; O(1)
    • For while (i > 1) {O(log n)
since i is halved in inner loop
        • For int j = 1; O(1)
        • For while (j < i) { O(log n)
since j is doubled inside loop
            • For j *= 2; O(1)
              }
            • For i /= 2; O(1)
              }}

```

Conclusion:As we have a nested loop with each having time complexity

then total time complexity for this code: $O(\log n) * O(\log n) =$
 $O(\log n)^2$

Problem 2 : Updated Binary Search

Algorithm Objective:

This implementation modifies the standard binary search algorithm to efficiently search within rotated sorted arrays.

The key enhancement is detecting the sorted portion of the array at each step to guide the search direction. Additionally, duplicate values are handled by shrinking the search range when ambiguity occurs.

Performance Analysis (Time Complexity):

- Best Case: $O(1)$

Target found at the first midpoint.

- Worst Case: $O(n)$

Occurs with heavy duplicates, making the search linear.

- Average Case: $O(\log n)$

Standard performance for rotated arrays.